1. A **web portal** for covered entities and administrators.
2. A **client-side browser module** that preprocesses and de-identifies claims before upload.
3. A **claims ingestion API** that accepts only the de-identified payload.
4. A **normalization and validation pipeline** based on NCPDP-derived fields and your canonical schema.
5. A **matching and reconciliation engine** that compares de-identified claim records to manufacturer/rebate-side data.
6. A **policy engine** that applies manufacturer-specific rules and contract-pharmacy designations.
7. An **append-only audit and ledger trail** for every upload, transformation, match decision, and manual override.
8. An **operations/reporting layer** for exports, disputes, compliance views, and manufacturer-facing dashboards.

The architecture I would build

1. Frontend

Build a browser app with two major areas: a portal and a local preprocessing/upload module.

The portal should support registration with a work email, covered-entity lookup, invitations, preferences, designation workflows, and an upload UI. That is consistent with the public registration snippets and homepage language.

The preprocessing module should run fully client-side and do four things before upload:

- parse the source Excel/CSV
- map incoming columns to the required field schema
- drop disallowed/raw PHI fields
- hash the allowed sensitive fields with SHA-3 plus a salt scheme before transmission

The public terms confirm that browser-side extraction and SHA-3 salting are part of the platform's design.

Recommended frontend choice: React + TypeScript. That is not confirmed, but it aligns with the public job profile's emphasis on JavaScript and modern web engineering.

2. Identity and tenant model

Use a multi-tenant design with these entities:

- organization / covered entity
- user
- role
- manufacturer participation
- designated contract pharmacy
- wholly owned exemption request

You should model users as belonging to one covered entity, with admin users able to invite others and manage designations. That is directly suggested by the public account setup and designation flows.

3. Upload and ingestion layer

Accept uploads only after client-side preprocessing. Store three objects for every submission:

- the original upload artifact metadata
- the de-identified claim payload
- a schema-validation report

Do not store raw unprocessed PHI on the server if your goal is to mirror the public model closely. The terms say the browser software excludes other data elements and the server receives the de-identified claims data for further processing.

For cadence, support scheduled reminders and due windows for the 1st and 16th submission cycle, since multiple manufacturer notices reference that pattern.

4. Canonical claim schema

Build a canonical claim schema based on the NCPDP field-number approach referenced in the Terms, rather than letting each upload format drive your backend schema. The NCPDP reference guide also notes that 340B determination usually relies on data from covered entity, wholesaler/manufacturer, and pharmacy, and that the determination may happen days or weeks after adjudication because of data availability and integrity controls. That supports a normalized, asynchronous reconciliation design rather than simple synchronous request/response matching.

At minimum, your canonical record should include:

- upload batch ID
- covered entity ID
- pharmacy ID
- product/NDC
- quantity
- service / adjudication / fill-related dates
- hashed prescription key
- payer type and rebate channel indicators
- manufacturer ownership/marketed-product mapping
- source file lineage fields
- validation status

5. Reconciliation and matching engine

This should be the center of the platform.

Publicly, the platform combines covered-entity claims data with rebate data to identify ineligible rebates, and its planned IntegriChain integration emphasizes retrieval, validation, and reconciliation at scale. So your engine should do at least four stages:

- candidate retrieval
- validation
- duplicate discount detection
- policy decisioning

A good reconciliation flow is:

Step A: Candidate retrieval
Fetch potentially related manufacturer-side or rebate-side records by product, covered entity, pharmacy, time window, payer type, and hashed claim keys.

Step B: Validation
Check required field presence, date-window rules, product eligibility, pharmacy designation status, and manufacturer policy applicability.

Step C: Duplicate-discount logic
The HRSA duplicate discount prohibition is the domain reason this exists: manufacturers need mechanisms to avoid a situation where a drug receives a 340B discount and a Medicaid rebate on the same unit.

Step D: Result classification
Classify every record as:

- exact match
- probable match
- unmatched
- duplicate / ineligible
- invalid data
- excluded by policy
- pending external data

You should store the reasoning for each classification, not just the final status.

6. Policy engine

340B ESP is publicly positioned as an end-to-end solution for defining and administering 340B policies, and the terms say the data may be analyzed for compliance with participating manufacturers' policies. Manufacturer notices also show policy-specific features like single contract pharmacy designations and exemptions.

So build a policy engine with:

- manufacturer
- effective date range
- included product list / NDC scope
- covered entity eligibility rules
- allowed contract pharmacy count
- exemption rules
- submission cadence expectations
- look-back windows

- output routing rules

This should be data-driven, not hard-coded.

7. Audit trail and ledger

Even if Second Sight does not publicly call it a “ledger,” you should build one because accuracy and defensibility matter here.

Use an append-only event log with records like:

- user registered
- designation changed
- upload received
- schema validated
- claim normalized
- candidate match generated
- policy result assigned
- manufacturer report exported
- manual override applied

The reason to do this is that public materials emphasize data accuracy responsibilities, de-identification method, reporting to manufacturers/payers/state agencies, and security incident handling. That is a classic audit-heavy system.

8. Reporting and exports

Publicly, the platform may report ineligible rebates and other derived platform data to participating manufacturers, payers, rebate processors, and state Medicaid agencies. So you need output pipelines, not just dashboards.

Build exports for:

- manufacturer policy compliance
- duplicate discount findings
- submission completeness
- unresolved exceptions
- designation and exemption status
- batch-level reconciliation results

Recommended stack

The most evidence-based stack is:

- **Frontend:** React + TypeScript
- **Portal/API app:** Ruby on Rails
- **High-throughput workers / reconciliation engine:** Go
- **Database:** PostgreSQL for transactional core, MongoDB only if you truly need flexible ingest/audit documents

- **Object storage:** Azure Blob Storage
- **Containers/orchestration:** Docker + Kubernetes
- **Cloud:** Azure
- **CI/CD:** GitHub Actions or Azure DevOps
- **Queueing:** Azure Service Bus or RabbitMQ
- **Search/reporting:** PostgreSQL materialized views first; only add Elasticsearch/OpenSearch if truly needed

The language and infrastructure choices are grounded by BRG's public hiring profile; PostgreSQL and Service Bus are my engineering recommendation, not publicly confirmed stack details.

Data model I would use

You asked for detail, so here is the core schema.

tenant tables

- covered_entities
- users
- roles
- invitations
- pharmacies
- designations
- exemptions

upload tables

- upload_batches
- upload_files
- schema_validation_results
- claim_rows_raw_deidentified

normalized core

- claims
- claim_products
- claim_parties
- claim_dates
- claim_hash_keys

policy

- manufacturers
- manufacturer_products
- policies
- policy_versions
- policy_rules
- policy_assignments

reconciliation

- rebate_records
- reconciliation_jobs
- candidate_matches
- match_decisions
- exceptions

audit / ledger

- audit_events
- export_runs
- access_logs
- manual_override_events

How to build the browser-side de-identification

This is the part that most people get wrong.

A close public-compatible design would be:

1. User uploads CSV/XLSX in browser.
2. Browser maps columns to required fields.
3. Browser strips any fields outside the allowed schema.
4. Browser computes salted SHA-3 hashes for prescription number and the date fields referenced publicly.
5. Browser serializes only the allowed de-identified payload and uploads that JSON/CSV to your API.

Important caveat: the terms say “no party will have direct access to the salt key.” Public sources do **not** explain exactly how that key management works. That is one of the places where you cannot accurately reproduce their internal implementation. The safest approximation is to keep salt management inside a dedicated key-management service, with rotating secrets and restricted access, but that is a design choice, not a publicly verified fact about their system.

How I would implement the backend in Go

Use Go for the ingestion workers, normalization pipeline, and reconciliation engine.

A practical service split:

- `upload-api`
  receives de-identified batches and writes metadata
- `schema-worker`
  validates file format and canonical mapping
- `normalization-worker`
  turns incoming records into canonical claims

- `policy-worker`
  resolves which manufacturer policies apply
- `reconciliation-worker`
  matches claims against rebate-side or manufacturer-side records
- `reporting-worker`
  generates manufacturer and ops exports
- `audit-service`
  writes append-only events from every service

That split lines up with the public evidence that the platform ingests de-identified claims, processes them further on the server, analyzes them with rebate data, and supports reporting and policy administration.

Matching logic to start with

You previously asked about OVI matching and ledger processes. In this system, your matching engine should probably not start with vague heuristics. It should start with deterministic business rules:

- same manufacturer-product scope
- same covered entity / contract pharmacy relationship
- eligible payer channel
- valid date window
- non-exempt claim
- no prior duplicate resolution
- hashed identifiers consistent with candidate rebate record

Then add ranked candidates only where deterministic exact matching is impossible.

Security and compliance controls

Because the public model is built around de-identified claims and regulated healthcare rebate workflows, you should implement:

- browser-side PHI minimization
- encrypted transport and storage
- role-based access control per covered entity
- immutable audit logs
- key management for salts and signing keys
- separate production and support access paths
- downloadable evidence for each decision
- alerting for malformed uploads and repeated failed submissions

The exact compliance framework used by Second Sight is not public in the sources I found, so I would not claim SOC 2, HITRUST, or similar unless you independently verify that later.

What you should not try to copy

Do **not** assume:

- their exact database schema
- their exact match rules
- their exact salt-key design
- their exact internal APIs
- their exact manufacturer-specific policy logic
- their exact frontend framework

Those details are not publicly documented in the sources I found, so presenting them as fact would be inaccurate.

The most accurate way to build this

If your goal is “as close as public information allows,” I would do it in four phases.

Phase 1: Public-compatible MVP

Build:

- covered-entity registration
- user invitation
- designation workflow
- browser-side de-identification
- de-identified file upload
- schema validation
- basic reconciliation statuses
- twice-monthly submission reminders

Phase 2: Policy engine

Add:

- manufacturer-specific policy versioning
- product/NDC scoping
- exemption handling
- effective-date management
- export routing rules

Phase 3: Scaled reconciliation

Add:

- queue-backed worker pools
- bulk ingestion
- exception queues
- manufacturer-side reporting
- in-cycle duplication outputs inspired by the IntegriChain-style workflow

Phase 4: Hardening

Add:

- event sourcing / append-only audit
- replayable jobs
- K8s deployment on Azure
- fine-grained access logs
- disaster recovery and batch replay tooling

My honest bottom line

The **highest-confidence facts** are:

- it is a web portal for covered entities,
- claims are de-identified in the browser before upload,
- SHA-3 with salting is used on certain fields,
- the server processes de-identified claims further,
- data is analyzed with rebate data to identify ineligible rebates,
- policy administration is central,
- uploads commonly run twice monthly,
- BRG hires for JavaScript/Rails/Go and mentions Kubernetes, Docker, MongoDB, and Azure.

Everything beyond that is best treated as **informed architecture design**, not secret truth.

I can turn this into a **build spec** next: service-by-service architecture, PostgreSQL schema, Go package layout, API endpoints, upload format, and the browser-side hashing flow.

What you can realistically build in Go

1. Upload API

You can build the backend service that receives the **de-identified** file or JSON payload from the browser, validates auth, stores batch metadata, and queues the file for processing. Public materials indicate that 340B ESP's browser software preprocesses and de-identifies claims before upload, so a server-side upload API is a natural Go service to build.

You would learn:

- HTTP servers

- file uploads
- JSON/CSV handling
- auth middleware
- request validation
- cloud storage integration

2. Schema validation service

After upload, you can build a Go worker that checks:

- required fields
- data types
- allowed formats
- missing values
- invalid date windows
- malformed rows

This fits Go very well because it is batch-oriented and performance-sensitive.

You would learn:

- parsing CSV/XLSX-derived data
- validation pipelines
- error reporting
- batch job design

3. Normalization pipeline

This is one of the best Go projects you could do. Take messy incoming claim records and convert them into a clean canonical structure.

That means:

- mapping source columns to standard fields
- converting dates and quantities
- standardizing IDs
- dropping unsupported fields
- creating internal claim objects

You would learn:

- clean architecture
- domain modeling
- transformations
- testable service design

4. Browser-side hashing helper? Mostly not in Go

This part is the main exception. Public 340B ESP terms say the de-identification browser software runs locally in the browser and applies SHA-3 hashing with salting to certain fields before upload. That makes this piece more naturally a **JavaScript/TypeScript** feature, not a normal Go backend service.

Technically, you *could* experiment with Go compiled to WebAssembly, but that is not the first thing I would learn.

5. Reconciliation engine

This is probably the **best Go learning project** in the whole system.

You can build the logic that:

- reads normalized claims
- retrieves candidate rebate-side or manufacturer-side records
- applies matching rules
- flags duplicates
- classifies outcomes
- writes decisions

Public descriptions of the platform emphasize retrieving, validating, and reconciling claims at scale and identifying ineligible duplicate rebates, which is exactly the kind of backend engine Go is excellent for.

You would learn:

- business rule engines
- concurrency
- worker pools
- DB querying
- deterministic matching logic
- idempotent processing

6. Policy engine

You can absolutely build this in Go.

A Go policy service could:

- load manufacturer policy definitions
- apply effective date rules
- check covered-entity designation logic
- decide whether a claim is eligible, excluded, pending, or ineligible

Public material says the platform is used for defining and administering 340B policies and for checking compliance with participating manufacturers' policies.

You would learn:

- rule evaluation
- config-driven logic
- versioned policy handling
- separation of domain logic from infrastructure

7. Audit trail service

Very good Go project.

Build an append-only event service that records:

- upload created
- validation failed
- claim normalized
- match generated
- override applied
- export produced

This matches the audit-heavy nature of regulated data workflows and is a strong systems design exercise. Public terms emphasize submission handling, monitoring, security incidents, and platform processing controls, which all point toward strong auditability needs.

You would learn:

- event logging
- append-only design
- JSON payload storage
- observability

8. Reporting/export worker

You can build a Go worker that generates:

- CSV exports
- summary reports
- unmatched exception lists
- manufacturer-specific output files

Since public materials indicate reporting of derived data to manufacturers and others, this is a realistic subsystem.

You would learn:

- scheduled jobs
- file generation
- aggregation queries
- asynchronous workflows

9. Admin/internal API

Very doable in Go.

Build REST endpoints for:

- viewing batches
- viewing validation errors
- reviewing match results
- resolving exceptions
- exporting reports
- managing policy versions

You would learn:

- REST API design
- pagination
- filtering
- auth/role checks

10. Background job system

This is another excellent Go area.

You can build workers for:

- batch ingestion
- nightly reconciliation
- retry queues
- failed upload reprocessing
- scheduled export creation

This is exactly the kind of backend plumbing where Go shines.

You would learn:

- concurrency
- channels
- queue consumers
- retries
- idempotency
- distributed job patterns

What I would *not* prioritize in Go first

I would not start with:

- the full browser UI
- the client-side spreadsheet upload interface
- the browser hashing/de-identification layer
- rich dashboards

Those are usually better in React/TypeScript.

If you want to become strong for a role like this, your Go learning should mostly live in the **backend and processing pipeline**.

Best learning path in Go for this system

If I were structuring this for you, I would learn and build in this order:

Stage 1: Core Go backend basics

Build:

- simple HTTP API
- PostgreSQL connection
- CRUD endpoints
- file upload endpoint
- structured logging

This gives you the base.

Stage 2: Batch processing

Build:

- CSV parser
- validation service
- normalization worker
- job queue

This teaches practical data engineering in Go.

Stage 3: Reconciliation engine

Build:

- candidate selection
- match rules
- duplicate detection
- status classification

This is the most interview-relevant part.

Stage 4: Audit + reporting

Build:

- append-only audit events
- report exports

- admin endpoints
- retryable jobs

That makes the project feel production-like.

The best “Go-only” subsystems for you

If you want the shortest answer: the parts most worth building in Go are:

- **Upload API**
- **validation service**
- **normalization pipeline**
- **reconciliation / matching engine**
- **policy engine**
- **audit event service**
- **report generation worker**
- **background job runners**
- **admin/internal APIs**

If you want to become job-ready fast

The three highest-value Go projects from this whole architecture are:

1. **A batch file ingestion + validation API**
2. **A reconciliation engine that matches claims to rebate records**
3. **An append-only audit and reporting backend**

Those three would teach you most of the Go skills that are likely relevant here: APIs, data processing, concurrency, backend architecture, and correctness under complex business rules. BRG’s public job materials also emphasize designing, developing, debugging, reviewing code, and collaborating on scalable software, which aligns closely with building these backend services.

The strongest next step would be to pick **one concrete mini-project** and build it end to end. My recommendation is: **start with the reconciliation engine in Go**, because it is the most technically interesting and the most likely to impress in an interview.

----------FINAL IMPLEMENTATION----------

What the platform should do

Core user story

An admin uploads a de-identified claims batch. The system:

- accepts the batch
- validates schema and fields
- normalizes records into a canonical claim model
- retrieves candidate rebate/manufacturer records
- applies reconciliation rules
- applies policy rules
- classifies each claim
- writes audit events
- exposes results through APIs and exports

High-level architecture

Use a **modular monolith first**, with boundaries that could later become microservices.

That gives you advanced architecture without making your life miserable.

Major modules

- API Gateway / HTTP layer
- Auth and tenant management
- Batch upload and ingestion
- Validation engine
- Normalization engine
- Reconciliation engine
- Policy engine
- Audit / event ledger
- Reporting / exports
- Scheduler / background jobs

Recommended architecture style

Use:

- **Hexagonal / clean architecture**
- domain services
- repository interfaces

- worker-based async processing
- event-driven internal flow

That will sound strong in interview and teach you proper Go structure.

Suggested tech stack

Backend

- Go
- `net/http` or `chi`
- PostgreSQL
- Redis
- RabbitMQ or NATS
- Docker
- Testcontainers or Docker Compose for local setup

Nice additions

- `sqlc` or `pgx`
- `zap` or `slog`
- `testify`
- OpenAPI / Swagger
- Prometheus metrics
- Makefile

Service/module architecture

1. API layer

Responsibilities:

- accept uploads
- expose batch status
- expose reconciliation results
- expose policy results
- expose audit history
- trigger exports

Example endpoints:

- `POST /v1/batches`

- `GET /v1/batches/{id}`
- `GET /v1/batches/{id}/claims`
- `GET /v1/claims/{id}`
- `GET /v1/audit-events`
- `POST /v1/exports`

---

2. Auth and tenancy

Responsibilities:

- org / covered entity ownership
- user roles
- per-tenant access control

Core concepts:

- organization
- user
- role
- pharmacy
- manufacturer participation

You do not need full auth complexity. A simple JWT or mocked auth layer is enough.

---

3. Upload and ingestion module

Responsibilities:

- receive JSON/CSV payload
- store batch metadata
- persist raw de-identified rows
- enqueue validation job

Flow:

1. upload batch
2. create `upload_batch`
3. save rows
4. emit `batch.uploaded`

---

4. Validation engine

Responsibilities:

- required field checks
- date validation
- quantity validation
- NDC format validation
- duplicate row checks
- schema mismatch reporting

Output:

- per-row validation result
- batch validation summary
- invalid rows marked early

This is a great area for table-driven tests.

---

5. Normalization engine

Responsibilities:

- map incoming payload to canonical model
- parse dates
- standardize identifiers
- derive normalized fields
- persist canonical claims

Example:
Input:

- variable input fields from upload

Output:

- one strict `Claim` aggregate in DB

This is one of the strongest Go modules you can show.

---

6. Reconciliation engine

This is the heart of the system.

Responsibilities:

- retrieve candidate rebate/manufacturer records

- compare on deterministic keys
- apply time-window logic
- apply payer channel logic
- detect duplicate discount risk
- classify outcomes

Statuses:

- matched
- probable_match
- unmatched
- invalid
- duplicate_discount_risk
- excluded_by_policy
- pending_external_data

This module should be async and job-based.

---

7. Policy engine

Responsibilities:

- load policy versions
- evaluate manufacturer-specific rules
- enforce effective dates
- enforce product scope
- enforce contract-pharmacy rules
- determine claim eligibility

Make this **data-driven**, not hardcoded.

Structure:

- `policies`
- `policy_versions`
- `policy_rules`

This is a great place to show careful domain design.

---

8. Audit ledger

Responsibilities:

- append-only event records
- record every important action

- support traceability

Events:

- batch_uploaded
- row_validated
- claim_normalized
- candidate_found
- match_decided
- policy_applied
- export_generated
- manual_override_applied

This makes the system feel production-grade.

---

9. Reporting and exports

Responsibilities:

- batch summaries
- unmatched claims
- duplicate risk report
- policy exclusion report
- org-level dashboards

Outputs:

- JSON API responses
- CSV exports
- summary metrics

---

10. Scheduler / background jobs

Responsibilities:

- pick pending batches
- retry failed jobs
- generate scheduled reports
- mark stale pending reconciliations

This is where Redis/RabbitMQ helps.

Data flow

Upload flow

1. client uploads batch
2. batch metadata stored
3. de-identified rows stored
4. validation job queued

Validation flow

1. worker loads raw rows
2. validates each row
3. marks invalid rows
4. valid rows sent to normalization

Normalization flow

1. worker transforms valid rows
2. creates canonical claims
3. emits claim normalized events
4. queues reconciliation

Reconciliation flow

1. worker loads canonical claim
2. fetches candidate rebate records
3. applies deterministic matching
4. applies policy rules
5. stores decision and reason
6. emits audit event

Reporting flow

1. API or scheduled job requests export
2. reporting worker builds result set
3. generates CSV
4. stores artifact
5. writes audit event

Suggested domain model

Main entities

- User
- Organization
- Pharmacy
- UploadBatch
- UploadRow
- ValidationResult
- Claim
- RebateRecord
- CandidateMatch
- MatchDecision
- Policy
- PolicyVersion
- PolicyRule
- AuditEvent
- ExportRun

Example PostgreSQL schema shape

tenancy

- `organizations`
- `users`
- `roles`
- `user_roles`
- `pharmacies`

ingestion

- `upload_batches`
- `upload_rows`
- `validation_results`

normalized

- `claims`

reconciliation

- `rebate_records`
- `candidate_matches`
- `match_decisions`

policy

- `policies`

- policy_versions
- policy_rules

audit

- audit_events

reporting

- export_runs
- export_files

Project structure in Go

```
claims-platform/
  cmd/
    api/
      main.go
    worker/
      main.go
  internal/
    app/
      server.go
      config.go
    domain/
      claim/
        entity.go
        service.go
        repository.go
      batch/
        entity.go
        service.go
        repository.go
      reconciliation/
        service.go
        matcher.go
        classifier.go
      policy/
        service.go
        evaluator.go
      audit/
        service.go
    transport/
      http/
        handlers/
          batch_handler.go
          claim_handler.go
          audit_handler.go
        middleware/
    infra/
      db/
      queue/
      logger/
```

```
      storage/
    jobs/
      validation_job.go
      normalization_job.go
      reconciliation_job.go
      export_job.go
    tests/
  migrations/
  api/
    openapi.yaml
  deployments/
    docker-compose.yml
  Makefile
```

This is advanced enough to impress without becoming chaotic.

Internal architecture pattern

Each module should follow this shape:

Entity

pure domain data

Service

business logic

Repository interface

contract for persistence

Infra implementation

Postgres, Redis, queue, etc.

That lets you say in interview:

I separated transport, domain logic, and infrastructure so the core business rules are testable and not tied directly to HTTP or SQL.

That is a very strong answer.

What to implement first

Phase 1

- API server
- batch upload endpoint
- store batch in Postgres
- simple validation worker
- audit event write
- batch status endpoint

Phase 2

- normalization pipeline
- canonical claim persistence
- reconciliation engine with mock rebate records
- claim results endpoint

Phase 3

- policy engine
- rule versioning
- exclusion logic
- export endpoint

Phase 4

- queue-backed workers
- retries
- idempotency
- metrics
- integration tests

Important advanced concepts to include

1. Idempotency

If a batch is re-submitted or a job is retried, it should not create duplicate claims or duplicate decisions.

This is an excellent thing to talk about.

2. Asynchronous processing

Uploads should return quickly, while heavy work runs in background workers.

3. State machine

Your batch should move through states like:

- uploaded
- validating
- invalid
- normalized
- reconciling
- completed
- failed

4. Append-only audit trail

Never mutate audit history. Always add events.

5. Deterministic classification

Store both:

- final decision
- reason code

6. Config-driven rules

Policies should come from DB or config, not scattered `if` statements everywhere.

Example API design

POST /v1/batches

Upload claims batch

Response:

```
{
  "batch_id": "b_123",
  "status": "uploaded"
}
```

GET /v1/batches/{id}

Returns batch state and counters

GET /v1/batches/{id}/results

Returns reconciled claims

GET /v1/claims/{id}

Returns one claim with:

- normalized fields
- match decision
- policy result
- audit history

POST /v1/policies

Create or update a policy version

POST /v1/exports

Generate export for a batch

Example reconciliation logic

Start with deterministic rules only.

A claim matches if:

- same org
- same pharmacy
- same product/NDC
- same hashed key
- service date within allowed window
- quantity valid
- not excluded by policy

If all match:

- `matched`

If product matches but hashed key missing:

- `probable_match`

If no candidate:

- `unmatched`

If both 340B discount and rebate apply:

- `duplicate_discount_risk`

Example policy rules

A policy version can contain:

- manufacturer ID
- effective date
- product scope
- allowed pharmacy designations
- exemption flags
- allowed payer types
- lookback window

Then your evaluator checks a claim against the active rule set.

Testing architecture

This matters a lot.

Unit tests

Test:

- validators
- date rules
- NDC rules
- reconciliation classifier
- policy evaluator

Table-driven tests

Very Go-like and very good for interviews.

Integration tests

Test:

- batch upload API
- worker processing
- DB persistence
- queue flow

Handler tests

Use `httptest`.

What makes this advanced

Not just CRUD. It includes:

- async workflows
- domain logic
- background jobs
- state transitions
- rule engines
- auditability
- exports
- real testing strategy

That is exactly the kind of project that sounds credible in a Go interview.

How to present it in interview

Say:

I built a backend platform that simulates a regulated claims reconciliation workflow. The system ingests de-identified claim batches, validates and normalizes them asynchronously, reconciles them against rebate-side records, applies configurable policy rules, and records every important action in an append-only audit trail. I chose Go because the workload is I/O-heavy, batch-oriented, and benefits from clear concurrency and strong testability.

That is a very strong walkthrough intro.

If you want the version that is even better for interview polish

I would slightly narrow the full platform to this **resume-worthy architecture**:

Final recommended scope

- API service
- validation worker
- normalization worker
- reconciliation worker
- policy evaluator
- audit event service
- reporting/export module
- Postgres + Redis + RabbitMQ
- unit + integration tests

That is advanced enough without becoming impossible.

My honest recommendation

Build this as a **modular monolith with async workers**, not microservices.

That gives you:

- strong architecture
- manageable implementation
- easy local development
- good interview story

Microservices would make the explanation sound more “senior,” but the codebase would be harder to finish well.

If you want, next I’ll turn this into a **full build blueprint** with:

- exact tables
- exact API endpoints
- Go package layout
- worker responsibilities
- queue messages
- test plan
- 2-week implementation roadmap.

User

|

```
 v
Ingress / Load Balancer
 |
 v
API Service (Kubernetes Deployment + Service)
 |
 +--> PostgreSQL / Cloud SQL
 |
 +--> Queue
      |
      +--> Validation Worker
      |
      +--> Normalization Worker
      |
      +--> Reconciliation Worker
           |
           +--> Policy logic
           +--> Audit events
```

In plain English: what Kubernetes is

Think of Kubernetes as a **manager for your app containers**.

Without Kubernetes:

- you manually run your API
- manually run your worker
- manually restart them if they crash
- manually scale them

With Kubernetes:

- you tell it what you want running
- it keeps those pieces alive
- it replaces crashed containers
- it helps expose them on the network
- it helps you manage config and secrets cleanly

That is the simple idea. Deployments manage app workloads, Services expose them inside the cluster, ConfigMaps hold non-secret config, and Secrets hold sensitive values.

What I would add to your project

Final project with Kubernetes

Claims Reconciliation Platform on GKE

A user uploads a de-identified batch.
Your platform:

- stores the batch
- validates it
- normalizes it
- reconciles it
- applies policy rules
- stores audit events
- exposes results

Kubernetes runs the moving parts.

The version I recommend

Use **GKE Autopilot** on Google Cloud for this learning project.

Why:

- Google manages most of the node/infrastructure work
- you still learn Kubernetes objects and deployment workflow
- it is simpler than managing a full cluster yourself
- Autopilot is specifically designed so you focus on workloads while GKE manages underlying nodes and infrastructure. It also automatically upgrades nodes in Autopilot.

So for learning, this is the cleanest path.

Your architecture with Kubernetes

Main components

1. API service

This is the part users talk to.

It handles:

- upload batch
- get batch status
- get results
- request exports

Kubernetes objects:

- **Deployment**
- **Service**

---

2. Validation worker

Background process that:

- picks up uploaded batches
- checks rows
- records validation results

Kubernetes objects:

- **Deployment**

---

3. Normalization worker

Background process that:

- transforms valid rows into canonical claims

Kubernetes objects:

- **Deployment**

---

4. Reconciliation worker

Background process that:

- compares claims with rebate/manufacturer records

- writes decisions

Kubernetes objects:

- **Deployment**

---

5. Policy evaluator

This can either:

- live inside reconciliation worker at first
- or later become its own service

For now, keep it inside reconciliation logic.

---

6. PostgreSQL

Stores:

- organizations
- users
- batches
- rows
- claims
- decisions
- policies
- audit events

For learning, I would **not** run Postgres inside Kubernetes first. Use **Cloud SQL** instead. Kubernetes runs stateless app pieces much more comfortably than beginners running databases in-cluster.

---

7. Queue

Needed so the API can quickly accept work and background workers can process it later.

Good choice:

- Pub/Sub if you want GCP-native simplicity
- RabbitMQ if you want more classic queue learning

For your first version, RabbitMQ is easier to “see” conceptually.

Architecture diagram in plain English

```
User
  |
  v
Ingress / Load Balancer
  |
  v
API Service (Kubernetes Deployment + Service)
  |
  +--> PostgreSQL / Cloud SQL
  |
  +--> Queue
          |
          +--> Validation Worker
          |
          +--> Normalization Worker
          |
          +--> Reconciliation Worker
                    |
                    +--> Policy logic
                    +--> Audit events
```

What each Kubernetes thing means for your project

Pod

A Pod is the running unit in Kubernetes.

For you:

- one API pod
- one validation worker pod
- one normalization worker pod
- one reconciliation worker pod

You usually do not create Pods directly. You create a Deployment, and Kubernetes manages Pods for you. Deployments manage sets of Pods and rolling updates.

Deployment

A Deployment says:

“Keep this app running, and keep this many copies of it alive.”

Example:

- 2 API pods

- 1 validation worker
- 1 normalization worker
- 1 reconciliation worker

If one crashes, Kubernetes recreates it. That is one of the biggest wins.

Service

A Service gives a stable network endpoint to Pods. This is how you expose your API or let other workloads reach it consistently even if Pods change.

For you:

- API Service gets a Service object
- workers usually do not need public Services unless other components call them directly

ConfigMap

A ConfigMap stores normal config, not secrets. Kubernetes docs explicitly caution that ConfigMaps are not for confidential data.

For you:

- app environment
- queue name
- feature flags
- log level

Secret

A Secret stores sensitive data like passwords and tokens. Kubernetes defines Secrets for confidential data that should not sit in your app code or image.

For you:

- DB password
- API keys
- signing keys

Ingress

Ingress is the front door.

It lets external traffic reach your API service through one entry point.

For your project:

- user hits one URL
- Ingress routes traffic to API service

The best way to learn Kubernetes through this project

Learn it in layers.

Layer 1: containerise the Go app

Before Kubernetes, each service must run in Docker.

You will create:

- `api` image
- `worker` image

Layer 2: run locally with Docker Compose

This helps you understand:

- app
- db
- queue
- environment variables

Layer 3: move app pieces to Kubernetes

Then deploy:

- API Deployment + Service
- worker Deployments
- ConfigMaps
- Secrets

Layer 4: add production-style features

Later add:

- readiness/liveness checks
- autoscaling
- Ingress
- Kustomize overlays

Kustomize is supported by `kubectl` for declarative customization of manifests, which is useful when you want dev/prod variations.

The exact project architecture I'd teach you

Namespace

Use one namespace:

- `claims-platform`

Deployments

- `claims-api`
- `validation-worker`
- `normalization-worker`
- `reconciliation-worker`

Services

- `claims-api-service`
- maybe `rabbitmq-service` if you run RabbitMQ in cluster

External managed services

- Cloud SQL for PostgreSQL
- optionally GCS for exports

Config

ConfigMap:

- `APP_ENV=dev`
- `QUEUE_NAME=claims-jobs`

Secret:

- `DB_URL`
- `JWT_SECRET`

How the flow works with Kubernetes

Step 1

User uploads a batch to the API.

The API pod:

- saves batch metadata to Postgres
- saves rows
- pushes a validation job to the queue

Step 2

Validation worker pod pulls the job.

It:

- validates rows
- writes results
- pushes normalization job

Step 3

Normalization worker pod pulls that job.

It:

- normalizes rows into claims
- pushes reconciliation job

Step 4

Reconciliation worker pod processes claims.

It:

- compares with rebate records
- applies policy logic
- writes decisions
- writes audit events

This is exactly the kind of workflow Kubernetes suits: multiple containerized workloads, some public-facing and some background processors.

Why Kubernetes is helpful here

1. Separate responsibilities

Your API and workers do different jobs.

Kubernetes lets you deploy and scale them separately.

2. Reliability

If a worker crashes, Kubernetes restarts it.

3. Scaling

If uploads grow, you can scale API or worker replicas independently.

4. Cleaner architecture story

In interview, this sounds much stronger than “everything runs in one process.”

5. Real DevOps learning

You learn:

- containers
- manifests
- services
- config
- secrets
- deployment workflow

What you should not do

Do **not** start with:

- microservices across 10 repos
- service mesh
- Kubernetes operators
- running everything including DB inside K8s
- overly complex networking

That will slow you down.

Your learning roadmap

Phase 1: app first

Build the Go app locally:

- API
- worker
- Postgres
- queue

Phase 2: Docker

Containerise API and worker.

Phase 3: Kubernetes basics

Deploy:

- one Deployment
- one Service
- one ConfigMap
- one Secret

Phase 4: full project on GKE

Deploy:

- API
- validation worker
- normalization worker
- reconciliation worker

Phase 5: polish

Add:

- Ingress
- readiness probes
- autoscaling
- logs
- environment overlays

The Kubernetes concepts you need to know for interview

Memorise these in simple language:

Deployment

Keeps my app running and manages replicas/updates.

Service

Gives a stable way to reach Pods over the network.

ConfigMap

Stores non-secret configuration.

Secret

Stores sensitive config safely.

Pod

Smallest running unit, usually created by a Deployment.

Ingress

Routes external traffic into the cluster.

What to say in interview

I used Kubernetes because the project has multiple backend workloads with different responsibilities. The API accepts uploads, while validation, normalization, and reconciliation run asynchronously as separate workers. Kubernetes lets me deploy and scale those parts independently, manage configuration cleanly with ConfigMaps and Secrets, and improve resilience because failed Pods are automatically recreated.

That is a strong answer.

My recommendation for your stack

Use this:

- **GKE Autopilot**
- Go API
- Go workers
- Cloud SQL Postgres
- RabbitMQ or Pub/Sub
- ConfigMaps and Secrets
- Ingress
- Kustomize for env variants

Best next step

We should do this in order:

1. final folder structure
2. Dockerfiles
3. Kubernetes manifests
4. GKE deployment commands

5. local-to-cloud workflow

I can draft the **actual Kubernetes architecture and starter YAML files** next.